iMak AI Lab at AraSeg Shared Task 2026: Greedy-Selected Arabic Encoder Ensembles with Structural Constraints for Sentence Segmentation

Artiom Lipinski, Ilya Makarov

iMak AI Lab, AXXX
Moscow, Russia
Correspondence: artiom0706@gmail.com

Abstract

We describe the iMak AI Lab submissions to the closed track of the AraSeg Shared Task 2026 on Arabic sentence segmentation, covering all four subtasks: Paragraph-Aware (PA), No-Paragraph (NP), No-Punctuation Paragraph-Aware (NoPnx-PA), and No-Punctuation No-Paragraph (NoPnx-NP). Our system treats segmentation as binary token classification with pretrained Arabic encoders fine-tuned only on the 128K-token AraSeg training split. The single-model recipe encodes documents in overlapping 512-token windows and trains for three times more epochs than the standard schedule, which is the largest source of improvement in our pipeline. Per-subtask ensembles are then built by greedy forward selection over a pool of seven encoder families, several seeds, and multi-variant joint-training variants. Decoding applies deterministic structural constraints and a decision threshold tuned on the development set. Ensemble members and thresholds are selected on the development set only, and no feedback from the Blind Test set is used. On the Blind Test set, the systems reach F_1 scores of 94.1 (PA), 91.0 (NP), 89.5 (NoPnx-PA), and 85.8 (NoPnx-NP), above the official baseline on all four subtasks. We report ablations for each design decision, including several that did not help.

1 Introduction

Sentence segmentation is a standard preprocessing step in NLP pipelines. Arabic makes it difficult: punctuation conventions vary across genres and eras, and 39% of the sentence boundaries in the AraSeg corpus have no punctuation cue at all (Elkholy et al., 2026a). The AraSeg Shared Task 2026 (Elkholy et al., 2026b) varies the difficulty across four subtasks by withholding paragraph boundaries, punctuation, or both.

We participated in the closed track of all four subtasks. Closed-track systems may be trained only on the released AraSeg training split: 174 documents with 128K tokens, about 12% of the data behind the reference models of the corpus paper. At this scale, the question is how much can be extracted from a small supervised corpus. Our contributions are validated training and ensembling decisions: (i) the standard 5-epoch fine-tuning recipe undertrains at this scale, and 15 epochs add +2.9 to +5.6 F_1, the largest single improvement in our pipeline; (ii) among seven Arabic encoders, AraELECTRA is the strongest, with the largest margin on the no-punctuation subtasks; (iii) multi-variant joint training, i.e., concatenating all four variant views of the training documents, adds +0.7 to +0.9 F_1 without external data; (iv) greedy forward selection of ensemble members adds +0.5 to +2.0 F_1 over uniform averaging, and combines with development-tuned thresholds and three deterministic structural constraints.

2 Background

Shared task. AraSeg (Elkholy et al., 2026b) formulates Arabic sentence segmentation as binary token classification: given a whitespace-tokenized document, predict for every token whether a sentence boundary follows it. The four subtasks differ in the available surface cues. PA keeps paragraph breaks (literal newline tokens) and punctuation; NP drops paragraph breaks; NoPnx-PA drops punctuation; NoPnx-NP drops both. Evaluation uses boundary precision, recall, and F_1, macro-averaged over documents. Punctuation only partly determines boundaries: on the training split, the boundary probability is 0.96 after a question mark, 0.68 after a period, and 0.17 after a comma, and dot-leader runs in exam-style documents are a systematic false trigger. We competed in the closed track of all four subtasks.

Related work. Classic sentence-boundary detection treats the task as punctuation disambiguation,

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 927–932
October 28-29, 2026 ©2026 Association for Computational Linguistics

System (dev)	PA	NP	NoPnx-PA	NoPnx-NP
Rule baseline	67.8	55.5	45.2	6.8
CAMeLBERT 5ep	91.1	87.7	76.9	75.1
CAMeLBERT 15ep	93.9	91.2	82.3	79.2
Uniform ens. (15)	94.3	92.5	84.9	81.8
Best single	94.2	92.0	85.2	82.3
Greedy ens.	**95.1**	**93.5**	**86.9**	**84.5**

Table 1: Development-set F_1 progression (tuned thresholds, constraints applied). "Rule baseline" segments on punctuation/paragraph cues only; "Best single" is AraELECTRA.

	PA	NP	NoPnx-PA	NoPnx-NP
Members / threshold	7 / 0.45	4 / 0.45	5 / 0.45	4 / 0.50
Open Test F_1	94.1	92.6	86.7	84.5
Blind Test set				
Precision	94.4	93.8	87.7	86.4
Recall	94.4	89.3	92.8	86.4
F_1	**94.1**	**91.0**	**89.5**	**85.8**
Top system F_1	95.3	92.0	91.2	89.1
Official baseline F_1	93.9	88.2	87.0	80.7
Final rank	8/11	6/9	5/8	8/9

Table 2: Final closed-track systems and official results on the Blind Test set, with the winning score, the official baseline, and our rank per subtask. All configurations are selected and thresholded on the development set; Open Test scores use the same thresholds. Ranks count participating teams and exclude the organizers' baseline, which our systems exceed on every subtask.

from unsupervised models such as Punkt (Kiss and Strunk, 2006) to rule-based PySBD (Sadvilkar and Neumann, 2020), with cross-corpus evaluations in Read et al. (2012). Later multilingual work drops the punctuation assumption: Ersatz (Wicks and Post, 2021), WtP (Minixhofer et al., 2023), and SaT (Frohmann et al., 2024). The AraSeg corpus paper (Elkholy et al., 2026a) benchmarks these families together with fine-tuned Arabic encoders and LLMs; fine-tuned CAMeLBERT-MSA is the strongest option in all four conditions, and LLMs trail fine-tuned encoders by more than 15 F_1 on average. We follow this result and build on fine-tuned encoders. Our pool consists of BERT-style (Devlin et al., 2019) Arabic models: CAMeLBERT-MSA/Mix (Inoue et al., 2021), AraBERTv2 and AraBERT-large-v2 (Antoun et al., 2020), AraELECTRA (Antoun et al., 2021), and ARBERTv2/MARBERTv2 (Abdul-Mageed et al., 2021; Elmadany et al., 2023).

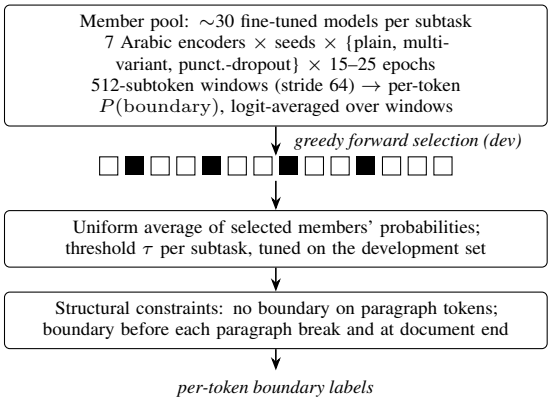


Figure 1: System pipeline for one subtask. Filled squares are the members kept by greedy forward selection (4–7 of ∼30).

3 System Overview

Figure 1 summarizes the system; the following paragraphs describe each stage.

Backbone. We fine-tune pretrained Arabic encoders with a binary token-classification head. Documents (median ≈620 tokens, maximum ≈21.6K)

are encoded as overlapping windows of 512 subtokens with stride 64; each word is represented by its last subtoken, and logits for words covered by multiple windows are averaged. Paragraph breaks map to a dedicated `[NL]` token. Two departures from the reference recipe matter. First, training length: the reference 5-epoch schedule undertrains at closed-track scale. With 15 epochs (best checkpoint by dev F_1, typically reached at epochs 11–14), a single CAMeLBERT-MSA gains +2.9 (PA) to +5.6 (NOPNX-PA) F_1 (Table 1). Second, multi-variant joint training (*mv*): all four variants are views of the same documents, so a model for subtask t can train on the concatenation of all four variants' training splits. This stays within the closed-track data and adds +0.7 to +0.9 F_1. A further augmentation samples random punctuation/paragraph-dropout views each epoch; none of its members enter the final ensembles.

Greedy ensemble selection. For each subtask we train a pool of ∼30 members that spans the seven encoders, several seeds, plain and mv variants, and 15- or 25-epoch schedules, and we ensemble by uniformly averaging per-token boundary probabilities. Averaging the full pool hurts: a single AraELECTRA beat a 15-model uniform ensemble on both no-punctuation subtasks, because weak members drag the average down. We therefore use greedy forward selection: start from the best single member, repeatedly add whichever member most improves tuned-threshold macro-F_1 on the selection data, and stop when no member helps. Selection and threshold tuning use the development set only; final configurations retain 4–7 members per subtask.

Decoding. The decision threshold is swept per subtask (0.15–0.80) on the development set. After thresholding we enforce three constraints that hold with 100% precision on train and dev: paragraph-break tokens never carry a boundary; the token before a paragraph break always does; the final non-break token of a document always does (Appendix B).

4 Experimental Setup

Table 3 summarizes the data. All models are trained only on the AraSeg training split (closed track); the development set was used for model selection and threshold tuning, and the open-test release only for reporting evaluation scores. The Blind Test sets contain 100 documents for the paragraph-aware subtasks and 212 for the no-paragraph ones. All members share one recipe (AdamW, lr 3×10^{-5}, batch 32, 15 epochs); full hyperparameters, tooling, and a note on an evaluation-script pitfall are in Appendix C. We report the official metrics: per-document boundary precision, recall, and F_1, macro-averaged over documents.

Split	Docs	Tokens	Bounds	Rate
Train	174	128,173	10,657	8.3%
Dev	222	163,609	12,985	7.9%
Open Test	262	158,626	12,509	7.9%
Blind Test	100/212	—	—	—

Table 3: AraSeg splits (token counts for the PA variant). Blind Test: 100 docs for PA/NOPNX-PA, 212 for NP/NOPNX-NP.

5 Results

Table 1 traces the development-set progression; Table 2 reports the final blind results and standings.

Final standings. Our systems finished 8th of 11 teams on PA, 6th of 9 on NP, 5th of 8 on NOPNX-PA, and 8th of 9 on NOPNX-NP, at 1.2, 1.0, 1.7, and 3.3 F_1 from the winning scores and above the official baseline on every subtask. The development-tuned thresholds carry a precision/recall imbalance onto the Blind Test sets (93.8/89.3 on NP, 87.7/92.8 on NOPNX-PA), which points to boundary-rate differences between the development and Blind Test set distributions. The winning closed-track PA score (95.3) matches the corpus paper's reference model trained on 8× the data (95.2 on open test), so fine-tuning recipes considerably better than the reference exist at this data scale. The following observations concern our own ablations.

Training length dominates. The gap between 5 and 15 epochs (+2.9 to +5.6) exceeds everything that ensembling adds afterwards, and best checkpoints landed at epochs 11–14 on every subtask. The corpus paper's reference model uses 5 epochs with 8× the data (Elkholy et al., 2026a); our results suggest that the appropriate schedule depends on data scale.

Selection beats averaging. On the development set, where selection is performed, greedy forward selection recovers +0.5 to +2.0 F_1 over uniform

averaging on all subtasks. AraELECTRA members appear in every final ensemble, in multi-variant form on the no-punctuation subtasks. AraBERT-large-v2 members are rarely the best singles but often enter selections, where they add diversity.

Negative results. A dynamic-programming decoder with a sentence-length prior gains up to +3.6 F_1 on weak single models but reduces to no-prior settings on selected ensembles. Punctuation-dropout augmentation contributed little.

6 Conclusion

We presented ensembles of Arabic encoders fine-tuned only on the 128K-token AraSeg training split, combined by greedy forward selection and decoded with development-tuned thresholds and structural constraints. The systems scored 94.1 / 91.0 / 89.5 / 85.8 F_1 on the Blind Test set of the four closed-track subtasks, above the official baseline in every case. In this data regime the effective decisions were simple: train three times longer than the standard recipe, use AraELECTRA, use the four variant views as additional training data, and select ensemble members greedily, since uniform averaging of the full pool was consistently worse. For the open track, a natural next step is to reconstruct the corpus paper's training data from its public sources, e.g., BAREC (Elmadani et al., 2025); we leave this to future work.

Limitations

We study only the closed track and encoder fine-tuning; we did not test LLM-based or genre-conditional approaches, which prior work reports as stronger on poetry. Member selection and threshold tuning used the development set, which adapts the configurations somewhat to its distribution. The ensembles require 4–7 encoder forward passes per document, several times the inference cost of a single model. Thresholds tuned on the development set transfer imperfectly: the precision/recall balance shifts on the Blind Test set, and our system has no mechanism to adapt the threshold to a new distribution without labeled data. The Blind Test set has no genre labels, so our error analysis is aggregate.

Acknowledgments

We thank the AraSeg organizers for a well-run task and their responsiveness to track-rule questions.

References

Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. 2021. ARBERT & MARBERT: Deep bidirectional transformers for Arabic. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7088–7105, Online. Association for Computational Linguistics.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, pages 9–15, Marseille, France. European Language Resource Association.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2021. AraELECTRA: Pre-training text discriminators for Arabic language understanding. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 191–195, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026a. Arabic sentence segmentation across genres and punctuation conditions. *Preprint*, arXiv:2606.08025.

Mohammed Elkholy, Khalid N. Elmadani, Nizar Habash, and Bashar Alhafni. 2026b. Arabic sentence segmentation shared task 2026. In *Proceedings of the Fourth Arabic Natural Language Processing Conference*, Budapest, Hungary. Association for Computational Linguistics.

Khalid N. Elmadani, Nizar Habash, and Hanada Taha-Thomure. 2025. A large and balanced corpus for fine-grained Arabic readability assessment. In *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria. Association for Computational Linguistics.

AbdelRahim Elmadany, ElMoatez Billah Nagoudi, and Muhammad Abdul-Mageed. 2023. ORCA: A challenging benchmark for Arabic language understanding. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 9559–9586, Toronto, Canada. Association for Computational Linguistics.

Markus Frohmann, Igor Sterner, Ivan Vulić, Benjamin Minixhofer, and Markus Schedl. 2024. Segment any

text: A universal approach for robust, efficient and adaptable sentence segmentation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11908–11941, Miami, Florida, USA. Association for Computational Linguistics.

Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. 2021. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 92–104, Kyiv, Ukraine (Virtual). Association for Computational Linguistics.

Tibor Kiss and Jan Strunk. 2006. Unsupervised multilingual sentence boundary detection. *Computational Linguistics*, 32(4):485–525.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*, New Orleans, Louisiana, USA.

Benjamin Minixhofer, Jonas Pfeiffer, and Ivan Vulić. 2023. Where's the point? self-supervised multilingual punctuation-agnostic sentence segmentation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7215–7235, Toronto, Canada. Association for Computational Linguistics.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, and 2 others. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, Vancouver, Canada.

Jonathon Read, Rebecca Dridan, Stephan Oepen, and Lars Jørgen Solberg. 2012. Sentence boundary detection: A long solved problem? In *Proceedings of COLING 2012: Posters*, pages 985–994, Mumbai, India. The COLING 2012 Organizing Committee.

Nipun Sadvilkar and Mark Neumann. 2020. PySBD: Pragmatic sentence boundary disambiguation. In *Proceedings of Second Workshop for NLP Open Source Software (NLP-OSS)*, pages 110–114, Online. Association for Computational Linguistics.

Rachel Wicks and Matt Post. 2021. A unified approach to sentence segmentation of punctuated text in many languages. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3995–4007, Online. Association for Computational Linguistics.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

A Final Ensemble Members

All members are token-classification fine-tunes trained for 15 epochs unless noted; *mv* denotes multi-variant joint training, *sk* the random seed.

PA (threshold 0.45). ARBERTv2 (s1); AraELECTRA (s1, s3); AraBERT-large-v2-mv (s2); ARBERTv2-mv (s1); AraBERTv2 (s2); MARBERTv2 (s2).

NP (threshold 0.45). ARBERTv2-mv (s1); AraBERT-large-v2-mv (s1); AraELECTRA (s3); CAMeLBERT-MSA (s3).

NoPnx-PA (threshold 0.45). AraELECTRA-mv (s1, s3); AraBERT-large-v2-mv (s1); ARBERTv2-mv (s1); ARBERTv2 (s1).

NoPnx-NP (threshold 0.50). AraELECTRA-mv (25 epochs, s1; 15 epochs, s3); AraBERT-large-v2-mv (s1); ARBERTv2-mv (s1).

B Structural Constraint Rules

The three post-processing rules, each of which holds with 100% precision on the training and development sets: (1) a paragraph-break token is never labeled as a boundary; (2) the token immediately preceding a paragraph break is always labeled as a boundary; (3) the last non-break token of every document is labeled as a boundary. Rules (1)–(2) apply only to the paragraph-aware subtasks; rule (3) applies everywhere. In the paragraph-aware subtasks, rule (2) alone accounts for roughly a third of all boundaries.

C Hyperparameters and Implementation

All members share one recipe: AdamW (Loshchilov and Hutter, 2019), learning rate 3×10^{-5} with linear decay and 10% warmup, batch size 32, windows of 512 subtokens with stride 64, bfloat16 mixed precision, 15 epochs (one member uses 25), cross-entropy loss on last-subtoken positions, best checkpoint by development F_1. Implementation uses HuggingFace Transformers (Wolf et al., 2020) and PyTorch (Paszke et al., 2019); a single member trains in well under an hour on one A100 GPU.

Evaluation-script pitfall. The official evaluation script reads the prediction column with pandas' default type inference, so a prediction string whose digits fit a 64-bit integer silently loses leading zeros or crashes the script. We evaluated locally with explicit string typing (`dtype=str`).